1 OFS Pad — Presentation Script and Technical Guide

1.1 How to use this document

This document contains two parts:

1. A complete speaking script and talking points for every slide in **OFS Pad — Architecture, Features & Everyday Use**.
2. A plain-English explanation of how the OFS Pad application is built, what each major code area does, how users move through the app, and how data and privacy are handled.

The explanation is based on the current production-connected repository:

- Production: https://ofs-pad-mqy1.vercel.app
- Repository: https://github.com/niyaztechs23/OFS-Pad
- Frontend: React, TypeScript, Vite, Tailwind-style utility classes
- Backend and authentication: Supabase
- Hosting and delivery: Vercel

---

2 Part 1 — Full presentation script

2.1 Opening — before Slide 1

Speaker script:

"Hello everyone. This presentation explains OFS Pad, a calm digital school workspace for students, teachers, and administrators. I will explain both what people can do inside the app and how the project is organised behind the scenes. The goal is to make the system understandable even if you are not a software developer. We will look at the student experience, the teacher workspace, privacy, the latest profile changes, and how a new version reaches the live application."

Key talking points:

- OFS Pad is not just a collection of pages; it is one connected school workspace.
- The main users are students, teachers, and administrators.
- The project uses a secure cloud database and a live web deployment.
- The presentation avoids unnecessary technical vocabulary and explains each technical part using everyday examples.

2.2 Slide 1 — OFS Pad: A calm digital school workspace

Purpose: Introduce the product and establish the simple message.

Speaker script:

"This is OFS Pad. The easiest way to understand it is as a digital school desk. A student can keep assignments, notebooks, class information, doubts, announcements, and planning tools in one place. Teachers can run classes, share work, grade submissions, answer questions, and publish updates. Administrators can control teacher approval and school-level access. The live production application is hosted at the Vercel address shown on the slide."

Talking points:

- The name OFS Pad suggests a workspace or notebook for learning.
- The supplied OFS Pad logo is used in the application, sign-in experience, browser icons, installed-app icons, and presentation.
- "Calm" means the interface is designed to reduce scattered information and keep the most important school tasks together.
- The application is available as a web app, so users can open it from a browser on a laptop or phone.

Transition:

"Now that we know what OFS Pad is, let us look at the everyday problem it is designed to solve."

2.3 Slide 2 — One place for the whole school day

Purpose: Show how OFS Pad brings common school tasks together.

Speaker script:

"Without a shared workspace, school information can become spread across messages, notebooks, separate files, and different platforms. OFS Pad brings the main activities into one place. Students can see assignments, keep notebooks, ask doubts, check announcements, view the planner, and join classes. The important idea is that these tools are connected to the student's classes instead of existing as unrelated pages."

Talking points:

- **Assignments:** Students can see work, deadlines, and submission status.
- **Notebooks:** Students can organise subject notes and notebook pages.
- **Doubt Wall:** Students can ask questions and, where appropriate, ask anonymously.
- **Announcements:** Teachers can communicate class updates.
- **Planner:** Exams and deadlines can be seen together.
- **Classes:** Students join using a teacher-provided code.

Simple explanation:

"Instead of asking, 'Which app or message contains this information?', a student can start in OFS Pad and find the relevant school task from the correct class."

Transition:

"The next slide explains how the software itself is divided into three understandable layers."

2.4 Slide 3 — How OFS Pad fits together

Purpose: Explain the architecture without jargon.

Speaker script:

"OFS Pad has three main layers. The first layer is what the user sees: screens, buttons, forms, dashboards, and pages. This is built with React and Vite. The second layer is the school's memory: accounts, profiles, classes, assignments, submissions, announcements, exams, and other records. That part is handled by Supabase. The third layer is the front door: Vercel gives the project its public web address and delivers the built application to users."

The everyday analogy:

- The **React/Vite interface** is the shopfront and the desks inside it.
- **Supabase** is the locked filing cabinet that stores the school records.
- **Vercel** is the public address and delivery service that makes the application reachable.

Talking points:

- React components create reusable interface pieces.
- Vite develops and builds the frontend quickly.
- Supabase provides authentication, database access, security policies, and realtime features.
- Vercel builds the source code and serves the resulting web files.
- The browser does not directly replace the database's security rules; the database remains the real authority.

Transition:

"Once a person signs in, the application needs to know which workspace they should see. That is the purpose of roles."

2.5 Slide 4 — One sign-in, the right workspace

Purpose: Explain authentication and role-based routing.

Speaker script:

"OFS Pad supports email and password sign-in and Google sign-in. After authentication, the application loads the user's profile and role. Students, teachers, and administrators then see different workspaces. A normal new account becomes a student. Teacher access is controlled through an approved email process, and administrator access is protected by database rules. This prevents the browser interface from being the only thing deciding who can see sensitive tools."

Talking points:

- Students can join classes, submit work, use notebooks, and ask doubts.
- Teachers can manage classes, assignments, grading, announcements, exams, and answers.
- Administrators can approve teacher access and manage school-level controls.
- Sessions persist in the browser and refresh automatically through Supabase Auth.
- Google OAuth uses the configured Supabase callback flow and returns the user to OFS Pad.
- The role is loaded from the profile and protected by backend policies.

Password visibility clarification:

"The eye button is available on sign-in and sign-up password fields. Passwords remain hidden by default, but a user can temporarily reveal the characters to check typing before submitting. The profile dashboard itself is now read-only after registration: it does not change the registered name or password."

Transition:

"Next, we follow a student through a normal day."

2.6 Slide 5 — A student's everyday journey

Purpose: Walk through the student workflow.

Speaker script:

“A student begins by joining a class with the code shared by the teacher. The student then sees the work that is due, writes notes in subject notebooks, submits assignments, asks questions on the Doubt Wall, and checks exams or deadlines in the planner. Each step is connected to the student’s account and class membership. This means the student is not looking at a generic public list; the student is seeing the school information relevant to their own classes.”

Talking points by step:

1. **Join a class:** The student enters a short code from the teacher.
2. **See what is due:** Assignments are loaded from the student’s class memberships.
3. **Write in notebooks:** Notes are grouped by subject and notebook.
4. **Ask a doubt:** The student can post a question and receive an answer.
5. **Check the planner:** Exams, deadlines, and upcoming tasks are easier to track.

Important user benefit:

“The workflow reduces the number of places a student needs to remember. The student can open OFS Pad and move from planning to learning to submission without leaving the workspace.”

Transition:

“The same connected design helps teachers manage the classroom from the other side.”

2.7 Slide 6 — What teachers can do

Purpose: Explain the teacher workspace.

Speaker script:

“Teachers use OFS Pad to create and organise classes, share assignments, grade submissions, provide feedback, answer doubts, post announcements, and schedule exams. The teacher does not need to collect all of these tasks in separate tools. The application also keeps teacher access scoped to the classes they own, so the teacher workspace is useful without exposing unrelated school data.”

Talking points:

- **Create a class:** The teacher creates the class and shares its join code.
- **Set assignments:** The teacher adds instructions and due dates.
- **Grade and feedback:** Students can see the result after the teacher records it.
- **Answer doubts:** Teachers can respond while preserving anonymous presentation where designed.
- **Post announcements:** Class information can be published in a shared feed.
- **Schedule exams:** Exam timing sits beside normal coursework.

Teaching benefit:

“The teacher can spend less time searching for scattered information and more time responding to student work.”

Transition:

“Convenience is important, but a school workspace also needs boundaries. The next slide explains how privacy is built into the system.”

2.8 Slide 7 — Privacy is built into the database

Purpose: Explain security in accessible language.

Speaker script:

"The screen can hide a button, but the database must also enforce the rule. OFS Pad uses Supabase Row Level Security, or RLS. In simple terms, each class behaves like a locked room. A student can read information for classes they joined. A teacher can manage classes they own. An administrator can perform protected approval operations. These rules are checked by the database, not only by the visual interface."

Talking points:

- Students should not be able to read another student's private work simply by changing a browser request.
- Teachers should not manage another teacher's classes.
- Ordinary users cannot promote themselves to teacher or administrator.
- Teacher approval is checked against the authenticated email and approval status.
- Sensitive service keys and OAuth secrets are not placed in the browser bundle.
- Realtime subscriptions are still subject to the database security model.

Plain-English summary:

"The interface decides what to show first. The database decides what is actually allowed."

Transition:

"The next slide covers the specific improvements requested for this release."

2.9 Slide 8 — What changed in this release

Purpose: Explain the profile dashboard, read-only identity details, password visibility, and branding.

Speaker script:

"This release adds a profile dashboard for every signed-in role. It shows the registered name, email, role, sign-in method, member-since information, role-specific statistics, classes, and quick links. The profile is intentionally read-only after registration: users cannot change their registered name, email, or password from this page. Password visibility remains available where it is useful—on sign-in and sign-up fields—so a user can check what they typed before submitting. Finally, the supplied OFS Pad logo is applied across the application and built brand assets."

Talking points:

- The profile is opened from the avatar in the top bar or the **Profile** sidebar item.
- Student statistics include classes joined, work due, submitted work, and graded work.
- Teacher statistics include classes taught, students reached, assignments, and work to grade.
- Administrator statistics show access level, account state, approvals, and session status.
- Full name and email are displayed as registered identity information.
- The sign-in and sign-up eye button does not store or change passwords; it only changes the field's visual type temporarily.
- The brand logo is used in the app, sign-in page, favicons, installed-app icons, and social preview.

Transition:

"After a code change is complete, it follows a simple release path to the real production app."

2.10 Slide 9 — How a change reaches the real app

Purpose: Explain the development and deployment path.

Speaker script:

"A change starts in the source code. It is committed to the main branch of the GitHub repository. Vercel detects the update, installs dependencies, runs the Vite production build, and serves the resulting files at the production address. React Router routes such as `/profile` still work when opened directly because the deployment rewrites application routes to the main HTML entry point."

Talking points:

1. **Edit the code:** A developer changes a page, component, hook, migration, or asset.
2. **Push to GitHub:** The main branch holds the source used for production.
3. **Vercel builds:** Vercel runs the project build and creates the `dist/` output.
4. **Live site updates:** Users receive the new application at the same URL.

Quality checks:

- TypeScript typecheck catches incorrect types.
- Lint catches common code-quality issues.
- The Vite production build confirms the app can be compiled.
- Focused smoke testing checks the affected workflow.
- Supabase migrations are applied separately when database structure changes.

Transition:

"We can now summarise the project in one sentence."

2.11 Slide 10 — OFS Pad in one sentence

Purpose: Close with the central value proposition.

Speaker script:

"OFS Pad is one secure, student-friendly place to learn, organise, and connect. It brings together student work, teacher workflows, and school-level controls while keeping the underlying data protected by database rules. The profile dashboard makes account information easy to understand, the authentication flow keeps access role-aware, and the production deployment makes the project available through one consistent web address."

Closing points:

- One place for schoolwork and communication.
- Different workspaces for students, teachers, and administrators.
- Security enforced in the backend, not just in the screen design.
- A read-only profile identity after registration.
- A maintainable path from source code to the live Vercel application.

Final sentence:

"Thank you. OFS Pad is designed to make the school day simpler for people while keeping the system disciplined behind the scenes."

---

3 Part 2 — OFS Pad technical guide in plain English

3.1 1. What the project is

OFS Pad is a single-page web application. The browser loads the application once and React changes the visible page as the user navigates. The project is not a collection of separate HTML files for every screen. It is a React application with routes such as `/auth`, `/student`, `/teacher`, `/admin`, and `/profile`.

The project has four practical parts:

1. **Frontend interface:** React components and pages.
2. **Data and authentication:** Supabase Auth and PostgreSQL data.
3. **Browser helpers:** hooks and small stores that load or save data.
4. **Deployment and brand assets:** Vite build output delivered through Vercel.

3.2 2. Main technologies

3.2.1 React

React lets the project describe the interface as reusable components. A page can be assembled from smaller pieces such as a card, badge, button, password field, top bar, sidebar, or notebook editor.

3.2.2 TypeScript

TypeScript adds type checking to JavaScript. It helps the project catch mistakes such as passing the wrong kind of value to a component or forgetting a required property.

3.2.3 Vite

Vite is the development server and production build tool. During development, it serves the app quickly. For production, it bundles the TypeScript and React code into optimised browser files in `dist/`.

3.2.4 Tailwind-style utility classes

The JSX uses utility classes for spacing, colours, layout, typography, borders, and responsive behaviour. For example, classes can describe a grid, a rounded card, a dark-mode colour, or a mobile breakpoint without creating a separate CSS file for every small visual change.

3.2.5 Supabase

Supabase provides:

- User authentication.
- The PostgreSQL database.
- Row Level Security policies.
- Database functions and triggers.
- Realtime subscriptions for selected features.

3.2.6 Vercel

Vercel hosts the production application. It builds the repository and serves the generated `dist/` directory. The production URL is `https://ofs-pad-mqy1.vercel.app`.

3.3 3. The most important source files

3.3.1 `src/App.tsx`

This is the top-level route map. It decides which page should open for a URL and which routes require authentication or a specific role.

Examples of responsibilities:

- Public landing page.
- Authentication page.
- Student routes.
- Teacher routes.
- Admin route.
- Shared profile route.

3.3.2 `src/pages/AuthPage.tsx`

This is the sign-in and sign-up screen. It handles email/password authentication, Google sign-in, role-aware teacher onboarding, validation, errors, and redirect behaviour.

The password field is hidden by default. The shared `PasswordField` component provides the eye button labelled **Show password** or **Hide password**. That button changes only the browser field type; it does not send a password anywhere by itself.

3.3.3 `src/components/auth/AuthProvider.tsx`

This component is the application's authentication coordinator. It restores the existing Supabase session, listens for authentication events, loads the matching profile, and exposes the current user information to the rest of the app.

A simplified flow is:

1. Supabase restores a saved browser session.
2. `AuthProvider` receives the session.
3. It loads the profile row associated with the user ID.
4. The app learns whether the user is a student, teacher, or admin.
5. Protected routes allow or deny access based on the role.

3.3.4 `src/pages/ProfilePage.tsx`

This is the shared read-only profile dashboard. It displays:

- Registered full name.
- Registered email identity.
- Role.
- Sign-in method.
- Member-since date.
- Last sign-in date.
- Role-specific statistics.
- Student or teacher classes.
- Quick links.
- Administrator access information where relevant.

The profile no longer contains controls to update the registered name or password. This is deliberate: registration establishes the identity details, and the profile page only displays them.

3.3.5 `src/data/nav.ts`

This file describes the navigation destinations for each role. It keeps sidebar and mobile navigation consistent with the user's workspace.

3.3.6 src/components/layout/

These components form the shared application frame:

- `AppShell.tsx` — common page shell.
- `TopBar.tsx` — top navigation, avatar, notifications, theme controls, and profile shortcut.
- `Sidebar.tsx` — desktop navigation.
- `BottomTabBar.tsx` — mobile navigation.

3.3.7 src/components/ui/

These are reusable visual building blocks:

- `Card.tsx` — consistent content containers.
- `Badge.tsx` — role and status labels.
- `Button.tsx` — common button styles.
- `PasswordField.tsx` — masked password input with the show/hide eye button.
- `SubjectTag.tsx` — subject labels.
- `StateMessage.tsx` — loading, empty, or error states.

3.3.8 src/hooks/

Hooks connect pages to data and interaction logic. Examples include:

- `useStudent.ts` — student classes, assignments, submissions, and student data.
- `useTeacher.ts` — teacher classes, assignments, grading queues, and teacher data.
- `useCommunity.ts` — doubts, answers, votes, and related updates.
- `useExams.ts` — exam data and exam workflows.
- `useNotebooks.ts` — notebook and page behaviour.

A hook keeps data-loading logic out of the visual page. The page asks for data; the hook knows how to retrieve and refresh it.

3.3.9 src/lib/supabase.ts

This creates the browser Supabase client using the public environment variables:

```
VITE_SUPABASE_URL
VITE_SUPABASE_ANON_KEY
```

Only the publishable anon key belongs in the browser. A service-role key must never be placed in frontend code.

3.3.10 src/lib/notebookStore.ts

This file defines the notebook data model and converts between the app shape and the database row. A notebook has an id, title, subject, a rough-note flag, a revision counter, timestamps, and a list of pages; each page holds typed text and pen strokes.

Two helpers keep the two storage layers in sync:

- `loadLocalNotebooks` / `saveLocalNotebooks` read and write a per-user browser cache. This keeps writing instant and lets the editor work offline.
- `notebookToRow` / `rowToNotebook` translate between the app object and a `public.notebooks` row, where pages are stored as JSON.

Older local notebooks used non-UUID ids, so `normaliseNotebook` upgrades them to real UUIDs on read. That is what allows a device notebook to be uploaded to Postgres without breaking ownership.

`src/hooks/useNotebooks.ts` performs the actual synchronisation:

1. Load the local cache so the UI can render immediately.
2. Fetch the signed-in student's rows from `public.notebooks`.
3. If the cloud is empty but the device has notebooks, upload them once.
4. If the device has notebooks the cloud does not, upload those too.
5. Debounce edits, write the local cache, then upsert to Postgres.
6. Report a `syncState` of `loading`, `syncing`, `synced` or `offline` so the interface can be honest about what happened.

3.3.11 `supabase/migrations/`

These SQL files define the database structure and security rules. They are applied in filename order. The migration set covers the core schema, submissions, notifications, roles, doubts, announcements, realtime publication, exams, teacher approvals, admin controls, and Google teacher-role protection.

3.3.12 `public/`

This folder contains public assets and route metadata, including:

- OFS Pad logo files.
- Browser and installed-app icons.
- `site.webmanifest`.
- `manus-routes.json`.

3.3.13 `vercel.json`

This configures the single-page application rewrite so direct navigation to routes such as `/profile` still loads the React entry point.

3.4 4. How a student signs in

The student opens `/auth` and chooses email/password or Google sign-in.

3.4.1 Email/password path

1. The user enters an email and password.
2. On signup, the user also enters the registered full name.
3. The form checks basic requirements, including the minimum password length.
4. Supabase Auth creates or verifies the account.
5. The app receives the session.
6. The profile row is loaded.
7. The user is redirected to the correct student route.

3.4.2 Google path

1. The user clicks Continue with Google.
2. Supabase starts the OAuth flow.
3. Google authenticates the user.
4. Google returns to the Supabase callback.
5. Supabase returns the authenticated session to OFS Pad.
6. `AuthProvider` loads the matching profile and role.
7. The app redirects to the correct workspace.

3.5 5. How roles work

A role is not meant to be trusted merely because a browser sends a value such as `role=teacher`. The real role is stored in the profile data and protected by database policies and controlled operations.

3.5.1 Student

A normal public signup becomes a student. Students can read and create data allowed by their class memberships and their own ownership.

3.5.2 Teacher

Teacher onboarding is invitation-only. An administrator approves a teacher email. The database checks that the authenticated email matches an approved, unused, unexpired approval before granting teacher access.

3.5.3 Administrator

Administrator access is controlled by the profile role and protected operations. The admin dashboard is used for teacher approval management and other school-level controls.

3.6 6. How the database is organised

The schema includes tables or concepts for:

- `profiles` — registered user identity and role.
- `classes` — teacher-owned classes.
- Memberships — which students belong to which classes.
- `assignments` — work created by teachers.
- `submissions` — student work and submission state.
- `notifications` — user-facing updates.
- `doubts` and votes — questions, answers, and community interaction.
- `announcements` — class updates.
- `exams` — scheduled tests or exam information.
- `notebooks` and notebook pages — student note-taking structures.
- Teacher approval and invitation records.

The exact data operations are implemented through Supabase queries, database functions, policies, and triggers.

3.7 7. How privacy works

Supabase Row Level Security is the main privacy boundary. It is useful to think of an RLS policy as a rule attached to a table that asks: “Is this signed-in user allowed to read or change this row?”

Typical rules include:

- A student can read data for a class they joined.
- A student can manage their own allowed work.
- A teacher can manage classes they own.
- An administrator can use protected approval operations.
- Ordinary users cannot update their own role.
- Teacher approval must match the authenticated email and status.

- Anonymous doubt presentation hides the author while preserving internal ownership and moderation information.

The frontend is not the final security boundary. Even if someone sends a custom request from outside the browser, Supabase policies should still reject data access that violates the rules.

3.8 8. How realtime updates work

Realtime is enabled for selected areas such as doubts, announcements, notifications, and submissions. A page subscribes when it opens and cleans up the subscription when it unmounts. The application also uses refresh or fallback behaviour so a missed websocket message does not permanently leave the screen stale.

In simple terms:

- A database change happens.
- Supabase can publish the change to an authorised client.
- The hook refreshes or updates the relevant page.
- The user sees the new information without a full browser reload.

3.9 9. What the read-only profile means

The profile dashboard is for understanding the account, not editing the registered identity.

The page shows:

- Full name entered during registration.
- Email used as the sign-in identity.
- Role.
- Sign-in provider.
- Member-since date.
- Last sign-in date.
- Role-specific counts and links.

The page does not provide:

- A name edit form.
- A save-name button.
- A password-change form.
- An update-password button.

Password visibility remains available on sign-in and sign-up because it helps the user check typing before submitting. Showing a password in a field is not the same as changing a password.

3.10 10. How the logo is used

The supplied logo is used in several places:

- Main application branding.
- Sign-in page.
- Landing page.
- Browser favicon sizes.
- Apple touch icon.
- Installed-app manifest icons.
- Social preview metadata.
- Presentation materials.

The public assets are stored under `public/ofs-pad-*.png`, and the manifest is `public/site.webmanifest`.

3.11 11. How to run the project locally

The normal local setup is:

```
npm install
cp .env.example .env
npm run dev
```

The `.env` file needs the public Supabase values:

```
VITE_SUPABASE_URL=https://<project-ref>.supabase.co
VITE_SUPABASE_ANON_KEY=<publishable-anon-key>
```

Never place these secrets in source control:

- Supabase service-role key.
- Google OAuth client secret.
- Database password.
- Vercel token.
- Personal account password.

3.12 12. How to check the project before release

The project uses these checks:

```
npm run typecheck
npm run lint
npm run build
```

3.12.1 Typecheck

TypeScript checks the relationships between components, hooks, data structures, and function arguments.

3.12.2 Lint

Lint checks common code-quality and React problems. The repository may still report non-blocking warnings documented in the README.

3.12.3 Build

The Vite build compiles the production browser bundle into `dist/`. A successful build means the source can be packaged for deployment.

3.13 13. How deployment works

The production path is:

1. A change is made in the working tree.
2. The checks are run locally.
3. The change is committed to GitHub `main`.
4. Vercel detects the new commit.
5. Vercel installs dependencies and runs `npm run build`.
6. The built `dist/` directory is served.

7. `vercel.json` rewrites browser routes to the React entry point.
8. The production URL serves the new version.

If a database schema change is needed, it should also be represented by a new forward-only Supabase migration and applied to the intended Supabase project. Frontend-only changes, such as removing profile controls, do not require a database migration.

3.14 14. Current release summary

The current release includes:

- Shared profile dashboard for students, teachers, and administrators.
- Read-only registered name and email display.
- Role-specific profile statistics.
- Quick links to the relevant workspace.
- Sign-in and sign-up password visibility buttons.
- Signup password minimum validation.
- Supplied OFS Pad branding throughout the application.
- Vercel production deployment from the GitHub main branch.

The profile intentionally does not include name-change or password-change controls after registration.

---

4 Short version for a non-technical audience

OFS Pad is a school workspace with three parts:

- **The screens:** what students, teachers, and administrators click.
- **The school memory:** the secure database that stores accounts, classes, work, marks, and updates.
- **The public address:** the Vercel deployment that lets people open the app.

The profile page is now an account information dashboard. It shows the registered details and useful statistics, but it does not allow anyone to change the registered name or password from the profile. The password eye button remains on the sign-in and sign-up fields so users can check what they typed before submitting.